Guia de Reprodutibilidade dos Experimentos
Diagnóstico Geométrico e Detecção de Anomalias no HAI 22.04

Versão 1.0

Wagner Carlos Mariani

Outubro de 2026

Sumário

1 Descrição geral do experimento **3**
- 1.1 Dataset HAI 22.04 e processos industriais . . . 3
- 1.2 Divisão temporal, guarda e normalização . . . 4
- 1.3 Autoencoder recorrente e alinhamento das janelas . . . 5
- 1.4 Representação residual comum . . . 6
- 1.5 Scores global e local de Mahalanobis . . . 6
- 1.6 Calibração normal-only e limiares . . . 7
- 1.7 Diagnóstico geométrico com dados normais . . . 7
- 1.8 Comparação principal e análise exploratória . . . 8

2 Organização dos diretórios e artefatos **8**
- 2.1 Repositório GitHub . . . 8
- 2.2 Dados originais e intermediários externos . . . 9
- 2.3 Resultados de referência publicados . . . 10

3 Roadmap para reprodução **11**
- 3.1 Preparação do ambiente . . . 11
- 3.2 Criar uma cópia segura e preservar os resultados . . . 11
- 3.3 Fluxo A: reproduzir o diagnóstico e os testes com os resíduos publicados . . . 12
- 3.4 Fluxo B: recuperar os resíduos do checkpoint publicado . . . 13
- 3.5 Fluxo C: novo treinamento da RNN . . . 14
- 3.6 Conferência de integridade e comparação das saídas . . . 14

4 Programas e scripts **15**
- 4.1 `rotular_testes_hai.py` . . . 15
- 4.2 `materializar_divisao_oficial_hai.py` . . . 15
- 4.3 `normalizar_hai.py` . . . 15
- 4.4 `rnn_hai_auditada.py` . . . 16
- 4.5 `extrair_residuos_monolitico.py` . . . 16
- 4.6 `recuperar_residuos_treino_normal_only.py` . . . 16
- 4.7 `diagnosticar_hai_residuos_normal_only.py` . . . 16
- 4.8 `avaliar_hai_mahalanobis_global_local.py` . . . 17
- 4.9 `analisar_hai_mahalanobis_por_grupo.py` . . . 17

5 Resultados de referência e interpretação **17**
- 5.1 Diagnóstico geométrico normal-only . . . 17

5.2 Avaliação principal com ataques 18
5.3 Análise exploratória dos detectores isolados 19

6 Integridade, limitações e reprodutibilidade 19
6.1 Registros SHA-256 e congelamento dos resultados 19
6.2 Separação entre dados normais e ataques 19
6.3 Limitações do alcance científico 20
6.4 Limitações operacionais da reprodução 20

1 Descrição geral do experimento

Este manual descreve a organização, o protocolo, os programas e os procedimentos de reprodução do experimento de detecção de anomalias realizado com o conjunto industrial HAI 22.04. O estudo examina, no espaço dos resíduos de reconstrução de uma única RNN-AE monolítica, a diferença entre uma avaliação global e uma avaliação por subespaços associados aos processos industriais.

O objetivo não é demonstrar que a divisão de variáveis seja sempre vantajosa. A motivação é investigar a possibilidade de que evidências localizadas apresentem menor contraste quando avaliadas em conjunto com muitas dimensões. A utilidade efetiva da decomposição depende da estrutura estatística dos resíduos, da partição considerada, das direções dos desvios e da regra de agregação dos escores.

Neste experimento, a comparação principal utiliza o score quadrático de Mahalanobis com covariância estimada por Ledoit–Wolf. Ambas as estratégias compartilham a mesma RNN e os mesmos vetores de resíduos. A diferença ocorre exclusivamente após a reconstrução, na formação dos escores de anomalia.

Este documento é um **manual técnico de reprodução**, destinado a acompanhar o repositório público, e não um capítulo da tese. Os resultados apresentados correspondem aos registros históricos auditados. O procedimento foi documentado e seus componentes foram verificados, mas uma execução integral de aceitação em uma instalação externa limpa permanece pendente.

1.1 Dataset HAI 22.04 e processos industriais

O HAI 22.04 contém seis arquivos originais de treinamento, denominados `train1.csv` a `train6.csv`, e quatro arquivos originais de teste, denominados `test1.csv` a `test4.csv`. Cada arquivo contém a informação temporal, 86 variáveis de processo e o indicador `Attack`.

As variáveis estão distribuídas entre quatro processos identificados pelos prefixos P1, P2, P3 e P4. A partição utilizada na análise dos resíduos é determinada por esses prefixos, e não por um procedimento de descoberta de grupos.

Tabela 1: Organização das 86 variáveis de processo.

Processo	Variáveis	Participação
P1	44	51,16%
P2	24	27,91%
P3	7	8,14%
P4	11	12,79%
Total	86	100%

O treinamento original contém 1.004.402 observações normais. Os quatro arquivos de teste contêm 361.200 observações, das quais 12.030 estão marcadas como ataque. A rotulação auditada identifica 58 eventos, distribuídos em 7 eventos no primeiro arquivo de teste, 17 no segundo, 10 no terceiro e 24 no quarto.

Os dados originais e a documentação da preparação podem ser obtidos em Google Drive — HAI 22.04. Sua disponibilidade e as condições de redistribuição devem ser verificadas antes de uma reprodução externa.

1.2 Divisão temporal, guarda e normalização

Foi adotada a divisão temporal A, fixada no arquivo `candidata_A_segmentos.csv`, preservado sob `dataset_trabalho/hai-22.04/logs/candidatas_treino_validacao/`. O treinamento utiliza uma porção temporal anterior de cada arquivo normal, e a validação utiliza uma porção posterior que não participa do treinamento.

Tabela 2: Divisão oficial dos 1.004.402 registros normais de treinamento.

Destino	Observações
Treinamento	806.046
Validação	198.000
Guarda	354
Resíduo não utilizado	2
Total	1.004.402

As 354 observações de guarda correspondem a 59 amostras entre treinamento e validação em cada um dos seis arquivos. Essa separação impede a sobreposição de janelas temporais de tamanho 60 entre as duas divisões. As janelas também não atravessam as fronteiras entre arquivos.

Nenhuma das 86 variáveis foi removida. As 17 variáveis booleanas foram preservadas em sua codificação binária. Para as demais 69 variáveis, foi aplicada normalização MinMax, com ajuste feito exclusivamente sobre as 806.046 observações do treinamento oficial. O mesmo transformador foi aplicado à validação e ao teste, sem novo ajuste e sem truncar valores que extrapolem o intervalo $[0, 1]$.

O diretório correto para a execução é `dataset_trabalho/hai-22.04/normalizados_oficiais/`. O diretório histórico `normalizados/` corresponde a uma preparação anterior à divisão independente e não deve substituir os conjuntos oficiais.

1.3 Autoencoder recorrente e alinhamento das janelas

Foi utilizada uma RNN-AE monolítica que recebe simultaneamente as 86 variáveis. A rede opera sobre janelas com 60 observações e utiliza 64 unidades ocultas, uma camada recorrente no encoder, uma no decoder, ativação recorrente `tanh` e uma projeção linear para reconstruir as variáveis originais. A função de perda de treinamento é o erro quadrático médio (MSE).

A configuração de referência utiliza batch de 512 janelas, Adam com taxa de aprendizado 10^{-3}, semente 42, *early stopping* com paciência de cinco épocas e limite de cem épocas. O *dropout* é zero. A validação é normal e serve ao acompanhamento da perda e à seleção do melhor checkpoint.

Para uma sequência individual com N observações, o número de janelas válidas é

$$N_{\text{janelas}} = N - 60 + 1 = N - 59. \tag{1}$$

Como cada score é associado ao último instante da janela, perdem-se 59 posições avaliáveis em cada arquivo, sem concatenar séries independentes. Os números efetivamente empregados pelos programas de diagnóstico e avaliação são:

Tabela 3: Observações e vetores de resíduos após o alinhamento temporal.

Conjunto	**Observações**	**Resíduos**
Treinamento normal (6 arquivos)	806.046	805.692
Validação normal (6 arquivos)	198.000	197.646
Teste (4 arquivos)	361.200	360.964

No teste alinhado permanecem 12.030 observações marcadas como ataque. O alinhamento retira observações iniciais normais de cada um dos quatro arquivos, mas preserva os 58 eventos identificados pelo protocolo.

1.4 Representação residual comum

Para cada instante avaliável, o programa extrai o vetor de reconstrução do último elemento da janela. Os resíduos assinados armazenados no pipeline seguem a convenção

$$\mathbf{e}_t = \widehat{\mathbf{x}}_t - \mathbf{x}_t \in \mathbb{R}^{86}, \tag{2}$$

na qual $\widehat{\mathbf{x}}_t$ é a reconstrução produzida pela RNN e $\mathbf{x}_t$ é a observação original. Os campos com prefixo `err_` representam esses resíduos com sinal; os campos `res_` correspondem a outra representação do erro e não são utilizados na modelagem Mahalanobis aqui documentada.

Os mesmos resíduos assinados alimentam tanto o detector global quanto os quatro detectores locais. **Não foram treinadas quatro redes distintas**. Assim, a comparação principal não mistura o efeito de dividir a entrada do encoder com o efeito de particionar a avaliação do vetor reconstruído.

1.5 Scores global e local de Mahalanobis

Seja $\widehat{\boldsymbol{\mu}}_G$ o vetor médio e $\widehat{\mathbf{P}}_G$ a matriz de precisão estimados por Ledoit–Wolf sobre os resíduos normais de treinamento. O score quadrático global é definido por

$$S_G(t) = \left(\mathbf{e}_t - \widehat{\boldsymbol{\mu}}_G\right)^{\top} \widehat{\mathbf{P}}_G \left(\mathbf{e}_t - \widehat{\boldsymbol{\mu}}_G\right). \tag{3}$$

Para cada processo G_k, o mesmo procedimento é aplicado ao subconjunto das dimensões daquele processo, produzindo $S_k(t)$ com média e precisão locais estimadas no treinamento normal. A agregação principal da estratégia local utiliza o máximo dos quatro scores:

$$S_L(t) = \max_{k \in \{\mathrm{P1},\mathrm{P2},\mathrm{P3},\mathrm{P4}\}} S_k(t). \tag{4}$$

Os modelos estatísticos finais são ajustados em `float64`. O arquivo `.npz` histórico, conservado no repositório, corresponde a uma referência numérica anterior de Mahalanobis, utilizada na conferência da recuperação dos resíduos, e não deve ser confundido com os novos ajustes `float64` do diagnóstico geométrico.

1.6 Calibração normal-only e limiares

Os limiares das duas estratégias são obtidos separadamente com o percentil 99 dos respectivos scores da validação exclusivamente normal:

$$\tau_G = P_{99}\big(S_G(\text{val})\big), \qquad \tau_L = P_{99}\big(S_L(\text{val})\big). \tag{5}$$

A decisão de anomalia utiliza a condição estrita $S(t) > \tau$. A regra de calibração resulta no mesmo número de alarmes na validação normal para as duas estratégias, mas **não** assegura taxas de falsos positivos idênticas no conjunto de teste.

Na execução registrada, os limiares são τ_G = 199,4393418878611 e τ_L = 135,92393580193848. Ambos produzem 1.977 alarmes nos 197.646 instantes de validação. Esses limiares permanecem congelados durante o teste; não há calibração com rótulos de ataque.

1.7 Diagnóstico geométrico com dados normais

O diagnóstico compara as margens de decisão e a geometria das matrizes de precisão global e locais, exclusivamente a partir dos dados normais. Definindo as médias dos scores de validação por $\overline{S}_G$ e $\overline{S}_k$, as margens são

$$\text{gap}_G = \tau_G - \overline{S}_G, \qquad \text{gap}_{L,k} = \tau_L - \overline{S}_k. \tag{6}$$

A razão entre margens do grupo k é

$$B_k = \frac{\text{gap}_G}{\text{gap}_{L,k}}. \tag{7}$$

Para examinar direções pertencentes ao subespaço G_k, a matriz de precisão global é restrita às respectivas dimensões, produzindo $\widehat{\mathbf{P}}_G^{(k)}$. Denotando a precisão estimada localmente por $\widehat{\mathbf{P}}_{L,k}$, calculam-se os autovalores generalizados positivos da relação

$$\widehat{\mathbf{P}}_G^{(k)}\mathbf{v} = r\,\widehat{\mathbf{P}}_{L,k}\mathbf{v}. \tag{8}$$

Sejam $r_{\min,k}$, $r_{\text{med},k}$ e $r_{\max,k}$ o mínimo, a mediana e o máximo desses autovalores. Os três fatores descritivos são

$$F_{\text{pess},k} = \frac{B_k}{r_{\max,k}}, \qquad F_{\text{med},k} = \frac{B_k}{r_{\text{med},k}}, \qquad F_{\text{opt},k} = \frac{B_k}{r_{\min,k}}. \tag{9}$$

O fator pessimista maior que 1 representa uma condição conservadora favorável à avaliação local para todas as direções examinadas, *dentro da aproximação quadrática*. Se esse fator não ultrapassa 1, o critério não fornece tal certificado. Isso não comprova inferioridade do detector local nem permite deduzir diretamente recall, precisão ou F1 em ataques reais.

1.8 Comparação principal e análise exploratória

Após o diagnóstico normal-only, os limiares congelados são aplicados ao teste com ataques reais do HAI. O experimento principal compara **Mahalanobis global** com **Mahalanobis local MAX**. A avaliação computa matrizes de confusão, métricas por amostra e detecção dos 58 eventos, sem ajuste de qualquer limiar no teste.

Uma análise posterior considera também os quatro detectores locais individualmente, cada um com seu próprio P99 da validação normal. Esses detectores isolados são **exploratórios**: seus resultados não devem ser combinados com a comparação principal como se fossem uma estratégia previamente selecionada.

2 Organização dos diretórios e artefatos

O repositório público reúne o código e os resultados de referência de menor tamanho, inclusive o arquivo de cabeçalho com a ordem oficial das 86 variáveis. Os CSVs originais e os grandes arquivos intermediários são obtidos separadamente pelo Google Drive. Os arquivos existentes na pasta publicada devem ser tratados como **referência imutável**; as reexecuções devem ocorrer em cópia independente.

2.1 Repositório GitHub

Endereço: github.com/Wagnermariani/hai-22-04-mahalanobis.

```
hai-22-04-mahalanobis/
|-- codigo/
|   |-- rotular_testes_hai.py
|   |-- materializar_divisao_oficial_hai.py
|   |-- normalizar_hai.py
```

```
|   |-- rnn_hai_auditada.py
|   |-- extrair_residuos_monolitico.py
|   |-- recuperar_residuos_treino_normal_only.py
|   |-- diagnosticar_hai_residuos_normal_only.py
|   |-- avaliar_hai_mahalanobis_global_local.py
|   |-- analisar_hai_mahalanobis_por_grupo.py
|   `-- requirements.txt
|-- dataset_trabalho/hai-22.04/logs/
|   `-- candidatas_treino_validacao/candidata_A_segmentos.csv
|-- resultado/
|   |-- rnn_hai/secaoMONOLITICO/ (checkpoint e referencia .npz)
|   `-- diagnostico_mahalanobis_normal_only/
|       |-- diagnostico/
|       |-- avaliacao_ataques/
|       `-- analise_exploratoria_por_grupo/
|-- documentacao/
|   |-- ordem_features_hai_22_04.csv
|   |-- manifest_artefatos_externos_sha256.txt
|   `-- manual_experimento_hai_22_04.pdf
|-- README.md
`-- .gitignore
```

2.2 Dados originais e intermediários externos

Os arquivos originais do HAI estão no Drive do dataset HAI 22.04. Os quatro artefatos grandes de reprodução estão no Drive dos artefatos de reprodução, reunidos no arquivo `HAI_22_04_artefatos_reproducao.zip`.

O ZIP preserva os caminhos relativos à raiz do projeto e contém exclusivamente:

```
resultado/diagnostico_mahalanobis_normal_only/
  residuos_treino_assinados.npy
resultado/rnn_hai/secaoMONOLITICO/
  residuos_monolitico/residuos_validacao_monolitico.csv
  residuos_monolitico/residuos_teste_monolitico.csv
  scores_rnn_secaoMONOLITICO.csv
```

O arquivo `residuos_treino_assinados.npy` contém 805.692 vetores assinados de 86 dimensões, armazenados em `float32`. Os dois CSVs de resíduos correspondem à validação e ao teste alinhados no tempo. O

arquivo de scores históricos da RNN é utilizado na auditoria do extrator e não substitui o cálculo de Mahalanobis em `float64`.

Ordem oficial das variáveis: os três programas finais consultam o pequeno arquivo `documentacao/ordem_features_hai_22_04.csv`, incluído no repositório e composto apenas pelo cabeçalho de 86 variáveis, sem linhas de observações. Ele foi extraído da ordem das colunas do treino normalizado oficial e conferido quanto ao número, à unicidade e à distribuição P1–P4. Os programas utilizam `pandas.read_csv(..., nrows=0)` para obter essa ordem. Assim, o **Fluxo A não exige a reconstrução dos CSVs normalizados**: bastam os resíduos externos, o arquivo de ordem e os artefatos versionados. Os Fluxos B e C continuam exigindo os dados originais e sua preparação.

2.3 Resultados de referência publicados

Os nove arquivos finais de resultados estão distribuídos nas três pastas a seguir:

```
resultado/diagnostico_mahalanobis_normal_only/
  diagnostico/
    diagnostico_por_grupo.csv
    diagnostico_resumo.csv
  avaliacao_ataques/
    metricas_global_local.csv
    comparacao_58_eventos.csv
    protocolo.json
  analise_exploratoria_por_grupo/
    metricas_por_detector.csv
    eventos_por_detector.csv
    intersecoes_com_global.csv
    protocolo_exploratorio.json
```

A pasta do checkpoint da rede é `resultado/rnn_hai/secaoMONOLITICO/`. Ela contém `modelo_treinado_rnn_secaoMONOLITICO.pth`; a subpasta `mahalanobis_global/` contém a referência `parametros_mahalanobis_global.npz`. Ambos os arquivos integram a versão publicada de referência.

3 Roadmap para reprodução

Esta seção apresenta o procedimento operacional. Os comandos devem ser executados a partir da raiz de uma **cópia de trabalho independente**. Os exemplos não foram ainda validados ponta a ponta em uma instalação limpa externa; o uso de `requirements.txt` pode exigir adequações de plataforma e disponibilidade de CUDA.

3.1 Preparação do ambiente

O ambiente de referência utilizou Python 3.10.12 em Linux. O arquivo `codigo/requirements.txt` registra versões do ambiente empregado, incluindo dependências associadas à GPU. Para preparar um ambiente virtual na cópia de execução:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r codigo/requirements.txt
```

O congelamento do ambiente facilita a rastreabilidade, mas não assegura instalação automática bem-sucedida em toda plataforma. Registrar alterações necessárias sem ocultar diferenças numéricas que possam afetar os hashes de referência.

3.2 Criar uma cópia segura e preservar os resultados

O primeiro cuidado é evitar executar os programas na pasta publicada, pois nela se encontram os resultados e o checkpoint históricos. O exemplo a seguir pressupõe que o clone esteja na pasta `hai-22-04-mahalanobis`, e que o terminal esteja no diretório-pai dessa pasta.

```
(
  set -e
  test ! -e hai_execucao && test ! -L hai_execucao || {
    echo 'ERRO: a pasta hai_execucao ja existe.'
    exit 1
  }
  cp -a hai-22-04-mahalanobis hai_execucao
  cd hai_execucao
  mkdir referencias_publicadas
```

```
  BASE_RESULTADO=resultado/diagnostico_mahalanobis_normal_only
  mv "$BASE_RESULTADO"/diagnostico referencias_publicadas/
  mv "$BASE_RESULTADO"/avaliacao_ataques referencias_publicadas/
  mv "$BASE_RESULTADO"/analise_exploratoria_por_grupo
   ↪  referencias_publicadas/
) && cd hai_execucao
```

Esse preparo deve ocorrer **somente na cópia recém-criada**. Os arquivos de saída da referência são deslocados para `referencias_publicadas/` dentro da cópia, preservando seus conteúdos; os diretórios correspondentes ficam disponíveis para novos resultados. Não repetir o procedimento sobre uma cópia de trabalho já utilizada.

3.3 Fluxo A: reproduzir o diagnóstico e os testes com os resíduos publicados

Este é o caminho de menor custo computacional. **Não é necessário baixar os dez CSVs originais do HAI nem executar a preparação/treinamento da RNN**. A cópia do repositório já inclui o mapa das variáveis em `documentacao/ordem_features_hai_22_04.csv`, o checkpoint histórico, os parâmetros `.npz` e as pequenas saídas de referência.

Baixar o ZIP do Drive de artefatos. Depois de conferir sua assinatura SHA-256, extrair os quatro arquivos preservando os caminhos relativos **dentro da cópia de execução**. O arquivo de scores históricos acompanha o pacote, mas não é consumido pelos três comandos deste fluxo. Por exemplo, supondo que o ZIP esteja disponível em `/caminho/para/HAI_22_04_artefatos_reproducao.zip`:

```
sha256sum /caminho/para/HAI_22_04_artefatos_reproducao.zip
unzip -n /caminho/para/HAI_22_04_artefatos_reproducao.zip -d .
sha256sum -c documentacao/manifest_artefatos_externos_sha256.txt
```

O valor esperado para o ZIP está indicado na Subseção 3.6. O parâmetro `-n` evita que um arquivo já existente seja sobrescrito durante a extração; a conferência dos hashes detecta eventual divergência de conteúdo. Não extrair na pasta de referência do GitHub.

Com os artefatos verificados, executar em sequência:

```
python codigo/diagnosticar_hai_residuos_normal_only.py
```

```
python codigo/avaliar_hai_mahalanobis_global_local.py
python codigo/analisar_hai_mahalanobis_por_grupo.py
```

O primeiro programa recalcula os limiares e fatores exclusivamente com resíduos normais. O segundo verifica os hashes exatos dos dois CSVs do diagnóstico, reproduz os limiares de validação e só então avalia os ataques. O terceiro confere a comparação principal antes de produzir as análises individuais por processo.

3.4 Fluxo B: recuperar os resíduos do checkpoint publicado

Este fluxo exige os dez CSVs originais do HAI, o mapa temporal oficial já incluído no repositório, o checkpoint da RNN e a referência `.npz`. Executar na cópia de trabalho, ainda sem os resíduos grandes nos respectivos caminhos de saída:

```
python codigo/rotular_testes_hai.py
python codigo/materializar_divisao_oficial_hai.py
python codigo/normalizar_hai.py
```

Os arquivos `train1.csv` a `train6.csv` e `test1.csv` a `test4.csv` devem estar em `dataset_trabalho/hai-22.04/` antes desses comandos. O mapa `candidata_A_segmentos.csv` já está presente no repositório e não precisa ser recalculado.

Em seguida, disponibilizar apenas o arquivo `scores_rnn_secaoMONOLITICO.csv` do ZIP de artefatos externos no caminho esperado e executar:

```
python codigo/extrair_residuos_monolitico.py
python codigo/recuperar_residuos_treino_normal_only.py
```

O primeiro programa extrai resíduos alinhados de validação e teste e compara o score de reconstrução de teste com a referência histórica da RNN. O segundo reconstrói resíduos assinados de treino, preservando verificações em relação aos parâmetros históricos do Mahalanobis. **Não extrair antecipadamente os três arquivos de resíduos do ZIP para os mesmos destinos**: o fluxo B pretende recriá-los e algumas etapas recusam sobrescrever arquivos existentes.

Após essa etapa, seguir o Fluxo A para reconstruir o diagnóstico e as avaliações.

3.5 Fluxo C: novo treinamento da RNN

O treinamento completo deve ocorrer em **uma terceira cópia independente**, nunca na referência publicada nem na cópia usada para verificar o checkpoint original. Após rotular, dividir e normalizar os dados, executar:

```
python codigo/rnn_hai_auditada.py
```

A execução cria novos pesos e escores. Mesmo com os mesmos hiperparâmetros e a semente registrada, diferenças de GPU, versões e operações numéricas podem produzir resultados distintos. Um novo treinamento constitui uma reprodução metodológica; não se deve alterar hashes, limiares ou comparações históricos para fazer os números coincidirem. As auditorias de identidade com o checkpoint e os parâmetros anteriores podem deixar de ser aplicáveis sem uma nova documentação de referência.

3.6 Conferência de integridade e comparação das saídas

Os resultados reproduzidos devem ser comparados com os nove arquivos conservados em `referencias_publicadas/` e com as tabelas apresentadas na Seção 5. As verificações de hashes devem falhar diante de resultados que não correspondam exatamente ao diagnóstico congelado. Nesse caso, investigar os dados, a ordem das variáveis e o ambiente antes de interpretar diferenças de métricas.

O ZIP externo possui a seguinte assinatura SHA-256:

```
d6b2200de3f7444778412870aed58744459e1346a7abd0680679bd21d589d300
```

Para conferir o arquivo baixado:

```
sha256sum HAI_22_04_artefatos_reproducao.zip
```

O manifesto SHA-256 dos quatro arquivos internos fica em:

```
documentacao/manifest_artefatos_externos_sha256.txt
```

Esse registro permite conferir os dados após a extração do ZIP, desde que os caminhos relativos à raiz do projeto sejam preservados. A conferência é feita com `sha256sum -c`, a partir da raiz da cópia de execução. Também é possível conferir que o pequeno CSV de cabeçalho lista exatamente 86 nomes distintos antes de executar qualquer análise; os próprios scripts verificam a partição completa em P1–P4.

4 Programas e scripts

Esta seção descreve os nove programas selecionados para o repositório final. Todos os comandos pressupõem que o terminal esteja na raiz do projeto e que o ambiente Python esteja ativo. A ordem corresponde ao pipeline experimental, não à ordem alfabética.

4.1 `rotular_testes_hai.py`

Lê os seis CSVs de treinamento e os quatro de teste do HAI. Preserva as colunas originais e cria arquivos derivados contendo a coluna `Tipo`, que identifica `Normal` ou um dos 58 eventos A101–A424. Os resultados são gravados em `dataset_trabalho/hai-22.04/rotulados/`; os CSVs originais não são alterados.

```
python codigo/rotular_testes_hai.py
```

4.2 `materializar_divisao_oficial_hai.py`

Aplica aos conjuntos rotulados o mapa fixado `candidata_A_segmentos.csv`. Materializa arquivos de treino, validação, guarda e resíduos não utilizados, respeitando a separação temporal. A seleção exploratória entre as estratégias candidatas de divisão não faz parte desta reprodução: reutiliza-se o mapa final publicado.

```
python codigo/materializar_divisao_oficial_hai.py
```

4.3 `normalizar_hai.py`

Ajusta o MinMax exclusivamente sobre a porção oficial de treinamento, preservando as 17 variáveis booleanas e mantendo as 86 variáveis de processo. Aplica a mesma transformação à validação e ao teste, sem calibrar parâmetros no conjunto de teste. Grava os arquivos finais em `dataset_trabalho/hai-22.04/normalizados_oficiais/`.

```
python codigo/normalizar_hai.py
```

4.4 rnn_hai_auditada.py

Define e treina a RNN-AE monolítica. Utiliza janelas temporais de 60 observações, dados normais e validação separada; registra o melhor checkpoint e gera escores de reconstrução. Esse programa é necessário apenas ao fluxo de novo treinamento e pode substituir o checkpoint da cópia de execução, razão pela qual não deve rodar na pasta de referência.

```
python codigo/rnn_hai_auditada.py
```

4.5 extrair_residuos_monolitico.py

Carrega o checkpoint publicado para extrair os resíduos assinados de validação e teste, alinhados ao último instante de cada janela. Também confere a saída de teste contra `scores_rnn_secaoMONOLITICO.csv`, necessário como entrada externa de auditoria. Os CSVs produzidos são utilizados posteriormente pelos detectores Mahalanobis.

```
python codigo/extrair_residuos_monolitico.py
```

4.6 recuperar_residuos_treino_normal_only.py

Reconstrói os resíduos assinados normais de treinamento e realiza uma conferência numérica com a referência histórica `parametros_mahalanobis_global.npz`. Produz `residuos_treino_assinados.npy`, consumido pelo diagnóstico final. A referência histórica não equivale a um novo detector treinado com os dados de teste.

```
python codigo/recuperar_residuos_treino_normal_only.py
```

4.7 diagnosticar_hai_residuos_normal_only.py

Utiliza somente resíduos normais de treinamento e validação. Ajusta Mahalanobis global e quatro locais com Ledoit–Wolf em `float64`; obtém os limiares P99 do global e do máximo dos grupos; calcula margens, autovalores generalizados e fatores geométricos. Escreve `diagnostico_por_grupo.csv` e `diagnostico_resumo.csv` exclusivamente em um diretório de destino ainda inexistente. Não consulta ataques.

```
python codigo/diagnosticar_hai_residuos_normal_only.py
```

4.8 `avaliar_hai_mahalanobis_global_local.py`

Valida os SHA-256 congelados dos dois arquivos de diagnóstico e reproduz os limiares na validação normal antes de abrir o teste. Em seguida, calcula os scores de ataque do Mahalanobis global e da fusão local MAX, produzindo métricas por amostra, eventos detectados e protocolo da avaliação. Os resultados são gravados em `avaliacao_ataques/`, sem sobrescrever os arquivos existentes.

```
python codigo/avaliar_hai_mahalanobis_global_local.py
```

4.9 `analisar_hai_mahalanobis_por_grupo.py`

Confere a reprodução da comparação principal e produz uma análise exploratória da participação dos processos na fusão MAX e do desempenho dos quatro detectores isolados, cada um com limiar próprio P99 na validação normal. Os resultados são separados em `analise_exploratoria_por_grupo/`, para evitar confusão com a comparação principal.

```
python codigo/analisar_hai_mahalanobis_por_grupo.py
```

5 Resultados de referência e interpretação

Esta seção resume os resultados registrados nos CSVs e JSONs do repositório. Os valores aqui apresentados são referências de auditoria, não resultados de uma nova execução realizada para escrever este manual.

5.1 Diagnóstico geométrico normal-only

Os fatores calculados por grupo são apresentados na Tabela 4. Em todos os processos o fator pessimista é inferior a 1. Assim, o diagnóstico **não produziu certificado conservador de vantagem local uniforme** nas direções consideradas. Isso não prova equivalência ou inferioridade dos detectores locais: a relação geométrica pode continuar a depender da direção efetiva da anomalia.

O diagnóstico foi produzido antes da consulta às observações de ataque, com os limiares global e local fixados nos resíduos da validação normal. Os fatores otimistas superiores a 1 em todos os processos indicam a possibilidade

Tabela 4: Fatores geométricos publicados por processo.

Processo	Pessimista	Mediano	Otimista
P1	0,001496	1,289977	3,761926
P2	0,256982	0,834728	1,067258
P3	0,000585	0,833331	1,120386
P4	0,000605	0,727775	1,194856

geométrica de direções favoráveis, mas não demonstram que tais direções correspondam aos ataques observados no teste.

5.2 Avaliação principal com ataques

A Tabela 5 apresenta a comparação principal sobre 360.964 instantes alinhados, sendo 12.030 de ataque e 348.934 normais. Os resultados de ambas as estratégias empregam os limiares definidos pela mesma validação normal, sem ajuste posterior.

Tabela 5: Métricas da comparação principal no teste HAI 22.04.

Métrica	Global	Local MAX
Verdadeiros positivos (TP)	5.981	3.214
Falsos positivos (FP)	3.794	1.852
Verdadeiros negativos (TN)	345.140	347.082
Falsos negativos (FN)	6.049	8.816
Recall	49,72%	26,72%
F1	0,548590	0,375994
FPR no teste	1,0873%	0,5308%
Eventos detectados	55/58	47/58

O método global detectou todos os eventos detectados pela fusão local MAX e mais oito eventos. A fusão MAX não identificou eventos exclusivos em relação ao global. O melhor recall global foi acompanhado de uma taxa de falsos positivos no teste mais alta. Embora o orçamento de alarmes tenha sido igualado na validação normal, **não houve pareamento do FPR observado no teste**; por isso, as diferenças de sensibilidade e falsos positivos devem ser interpretadas conjuntamente.

Três eventos não foram identificados por nenhuma das duas estratégias principais: A107, A209 e A212. Essas observações pertencem ao resultado

empírico e não foram utilizadas para reajustar os limiares congelados.

5.3 Análise exploratória dos detectores isolados

Na análise posterior, os detectores individuais P2 e P4, calibrados com seus próprios limiares P99 na validação normal, produziram alarmes em dois eventos não detectados pela comparação principal. Para A209, P2 registrou nove alarmes nas 163 observações de ataque e P4 registrou um. Para A212, P2 registrou cinco alarmes nas 56 observações de ataque e P4 registrou um.

Essa evidência mostra complementaridade pontual de detectores locais isolados, mas **não valida uma superioridade geral da modularização**. Os detectores P2 e P4 isolados também apresentaram, respectivamente, 8.585 e 5.282 falsos positivos no teste, e seus limiares diferem do limiar da fusão MAX. Portanto, essa etapa deve permanecer identificada como exploratória e não ser apresentada como confirmação prospectiva da hipótese.

6 Integridade, limitações e reprodutibilidade

6.1 Registros SHA-256 e congelamento dos resultados

Os dois arquivos do diagnóstico normal-only que fundamentam a avaliação principal possuem as assinaturas:

```
5d9713a1d89065585e66dc02dfb4a42646be646779b615c55430a29951430901
  diagnostico_por_grupo.csv
ac87726c478f6273eb4d5971828b41270cbfda0b61637d767801da5d04bdaf1a
  diagnostico_resumo.csv
```

Esses valores estão incorporados aos verificadores dos scripts de avaliação. Qualquer divergência deve ser investigada como possível diferença de entradas, ordem das variáveis, precisão numérica ou ambiente, sem alterar silenciosamente o protocolo publicado. Há, adicionalmente, um manifesto de quatro arquivos externos e um SHA-256 do arquivo ZIP que os reúne.

6.2 Separação entre dados normais e ataques

O modelo de reconstrução, os ajustes de covariância e os limiares globais e locais utilizam informações normais de treinamento e validação. O diagnós-

tico geométrico também é calculado *normal-only*. Os rótulos de ataque e a contagem dos 58 eventos entram somente na avaliação empírica do teste e nas análises exploratórias posteriores explicitamente identificadas.

6.3 Limitações do alcance científico

A condição geométrica examinada depende de uma aproximação quadrática, da matriz de precisão e das margens normais aos limiares. Não é um teorema de maior desempenho empírico para ataques arbitrários. O resultado do HAI 22.04, em particular, mostrou superioridade do Mahalanobis global sobre a fusão local MAX no número de eventos detectados, nas condições e nos limiares publicados.

O experimento não compara uma RNN monolítica com um conjunto de quatro RNNs especializadas. A partição ocorre somente nos resíduos e serve para isolar, tanto quanto possível, o efeito do operador de detecção local. Portanto, os resultados não devem ser extrapolados automaticamente para arquiteturas com múltiplos autoencoders.

A escolha dos quatro grupos utiliza a nomenclatura P1–P4 do HAI e não deve ser confundida com descoberta não supervisionada da partição por correlação. Além disso, a estratégia MAX e os detectores individuais possuem regras de calibração distintas, que não devem ser fundidas na interpretação dos resultados.

6.4 Limitações operacionais da reprodução

O repositório contém o checkpoint e as pequenas saídas de referência, mas não incorpora os CSVs originais e os resíduos volumosos. A reprodução exige a obtenção dos arquivos externos e o posicionamento correto nos diretórios documentados. A ordem das 86 variáveis está documentada em um CSV de cabeçalho versionado, eliminando essa dependência específica do Fluxo A. Já a recuperação a partir do checkpoint e um novo treinamento exigem os dez CSVs HAI originais e a reconstrução dos conjuntos normalizados. Os dados completos e o ZIP externo não são distribuídos pelo GitHub.

Os três scripts finais impedem a utilização de diretórios de resultados preexistentes. Por isso, deve-se sempre preparar uma cópia de trabalho e preservar os resultados da referência, sem executá-los diretamente na pasta publicada.

A repetição numérica estrita, em particular quando inclui um novo treinamento, pode depender da versão das bibliotecas e do hardware utilizado.